

Программа для определения типа ориентированного графа	
Программа и методика испытаний	
Студент	Иванов Д.В.
Преподаватель	Якимов Г. А.
Сдано	

1 Объект испытаний

Программа для определения типа ориентированного графа (его соответствия дереву)

Программа написана с помощью языков программирования С и С++

2 Цель испытаний

Целью испытаний является проверка соответствия программы требованиям к функциональным характеристикам

3 Требования к программе

Требования к программе изложены во внешней спецификации

4 Методы испытаний

Перечень проверок для `adjacencyList::substract` представлен в приложении 1

Перечень проверок для `parseGraphFromText` представлен в приложении 2

Перечень проверок для `DirGraph::createSpanningTree` представлен в приложении 3

Перечень проверок для `findGraphType` представлен в приложении 4

Приложение 1

Методика тестирования функции adjacencyList::substract

/**

* @brief вычитает из списка смежности другой список, возвращая разность

* @param subtrahend ссылка на вычитаемое

* @return ссылку на новый список смежности

*/

adjacencyList * substract(adjacencyList & subtrahend);

№	Название теста	Входные данные (уменьшаемое)	Входные данные (вычитаемое)	Ожидаемый результат
1	partialOverlap	{{1, 2}, {2}, {}}	{{1}, {}, {0}}	{{2}, {2}, {}}
2	equalLists	{{1}, {2}, {0}}	{{1}, {2}, {0}}	{{}, {}, {}}
3	noOverlap	{{1}, {}}	{{}, {0}}	{{1}, {}}
4	fullOverlapOfSubtrahend	{{1,2,3}, {1,2}, {}}	{{1,2}, {2}, {}}	{{3}, {1}, {}}
5	fullOverlapOfMinuend	{{1,2}, {2}}	{{1,2,3}, {2}}	{{}, {}}
6	emptyMinuend	{{}, {}, {}}	{{1}, {2}, {}}	{{}, {}, {}}
7	emptySubtrahend	{{1}, {2}, {}}	{{}, {}, {}}	{{1}, {2}, {}}
8	diffVertexCntMinSmaller	{{1}, {}}	{{1}, {2}, {0}}	{{}, {}}
9	diffVertexCntMinBigger	{{1}, {2}, {0}}	{{1}, {}}	{{}, {2}, {0}}

10	SelfSubtraction (один и тот же объект)	{{1, 2}, {0}}	{{1, 2}, {0}}	{{}, {}}
11	duplicateEdges	{{1, 1, 2}, {}}	{{1}, {}}	{{1, 2}, {}}
12	loopEdges	{{0, 1}, {2}, {}}	{{0}, {2}, {}}	{{1}, {}, {}}
13	complexTest1	{{0, 0, 1, 2}, {0, 2}, {2}}	{{0, 1}, {0, 1, 2}, {2}}	{{0, 2}, {}, {}}
14	complexTest2	{{0, 1, 1}, {0, 1, 2}, {0}}	{{1}, {1}}	{{0, 1}, {0, 2}, {0}}
15	complexTest3	{{1, 0, 0}, {2, 0, 1}, {0}}	{{0, 1, 1}, {1}, {1}}	{{0}, {0, 2}, {0}}

Приложение 2

Методика тестирования функции parseGraphFromText

/**

* @brief создаёт граф на основании текстового описания (вектора строк)

* @param fileText ссылка на текст для парсинга

* @param inDegrees ссылка на массив для подсчёта степеней захода каждой вершины

* @param errorsVector ссылка на вектор для возникших ошибок

* @return ссылку на созданный граф

*/

DirGraph * parseGraphFromText(std::vector<std::string> & fileText, int * inDegrees[1000] std::vector errorsVector);

№	Название теста	Входные данные	Ожидаемый результат
1	basicValidGraph	digraph G { 0 1 2 0->1 1->2 }	countOfVertices: 3 adjacencyList: {1}, {2}, {} inDegrees = [0,1,1]
2	isolatedVertices	digraph G { 0 1 2 3 0->1 }	countOfVertices: 4 adjacencyList: {1}, {}, {}, {} inDegrees = [0,1,0,0]
3	loops	digraph G { 0 1 2 0->1 1->1 2->2 }	countOfVertices: 3 adjacencyList: {1}, {1}, {2} inDegrees = [0,2,1]
4	multigraph	digraph G {	countOfVertices: 3

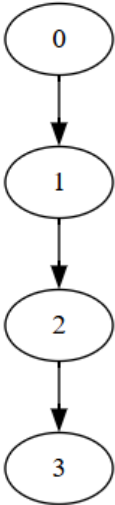
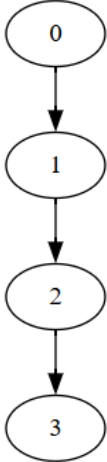
		<pre> 0 1 2 0->1 0->1 0->2 2->0 } </pre>	<pre> adjacencyList: {{1, 1, 2}, {}, {0}} inDegrees = [1,2,1] </pre>
5	noEdges	<pre> digraph G { 0 1 2 } </pre>	<pre> countOfVertices: 3 adjacencyList: {{}, {}, {} } inDegrees = [0,0,0] </pre>
6	emptyGraph	<pre> digraph G { } </pre>	<pre> countOfVertices: 0 adjacencyList: {} inDegrees = [] </pre>
7	whiteDelimeters	<pre> digraph G { 0 1 2 0 -> 1 1->2 } </pre>	<pre> countOfVertices: 3 adjacencyList: {{1}, {2}, {} } inDegrees = [0,1,1] </pre>
8	disconnectedComponents	<pre> digraph G { 0 1 2 3 4 0->1 1->0 3->4 } </pre>	<pre> countOfVertices: 5 adjacencyList: {{1}, {0}, {}, {4}, {} } inDegrees = [1,1,0,0,1] </pre>
9	defenitionLineSyntaxError	<pre> bigraph G { 0 1 2 3 4 0->1 1->0 3->4 } </pre>	defenitionLineSyntaxError
10	incorrectEdgeSyntax	<pre> digraph G { 0 1 </pre>	innerDescriptionSyntaxError

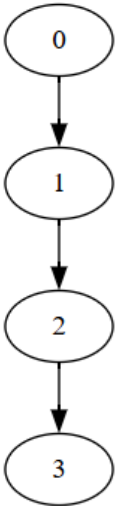
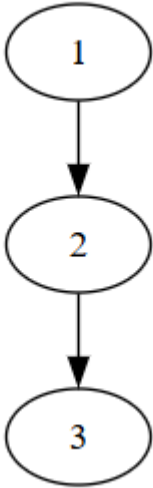
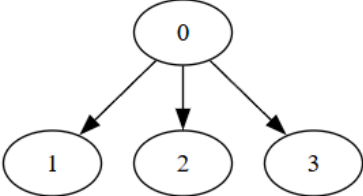
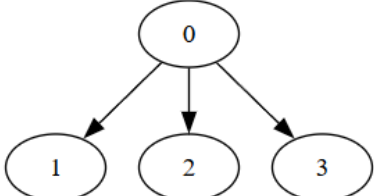
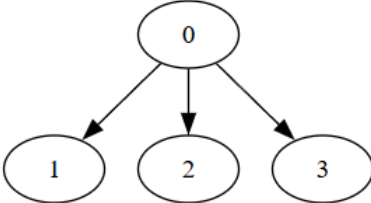
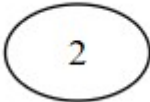
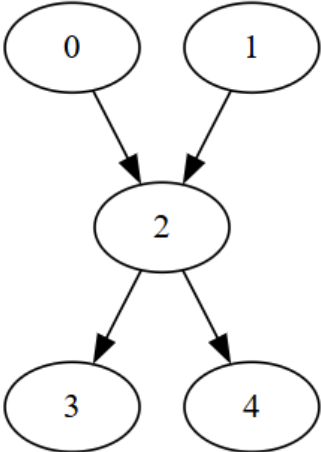
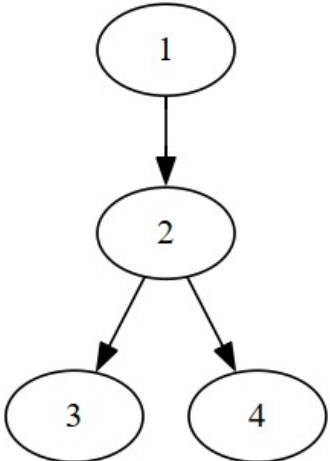
		<pre> 2 0--1 1->0 } </pre>	
11	incorrectVertexSyntax	<pre> digraph G { 0 1 a 0->1 1->0 } </pre>	innerDescriptionSyntaxError
12	endLineSyntaxError	<pre> digraph G { 0 1 2 0->1 1->0] </pre>	endLineSyntaxError
13	combinedErrors	<pre> bigraph G { 0 1 2 0--1 1->0] </pre>	defenitionLineSyntaxError innerDescriptionSyntaxError endLineSyntaxError
14	notStatedVertex	<pre> digraph G { 0 1 0->1 1->2 } </pre>	innerDescriptionSyntaxError
15	wrongDescriptionOrder	<pre> digraph G { 0->1 1->2 0 1 } </pre>	InnerDescriptionSyntaxError
16	severalInnerErrors	<pre> digraph G { 0 1 0-1 1-<0 } </pre>	InnerDescriptionSyntaxError InnerDescriptionSyntaxError

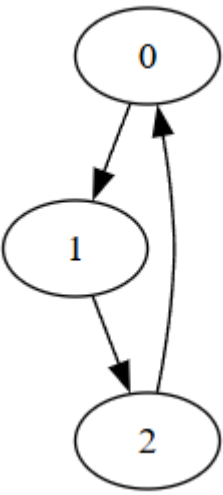
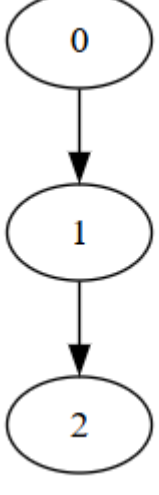
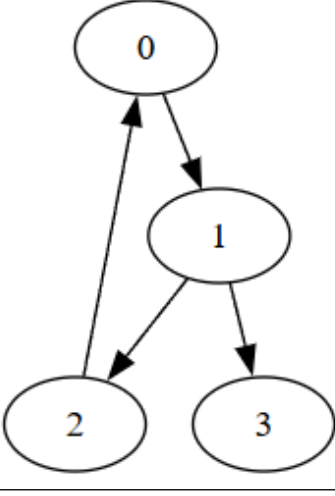
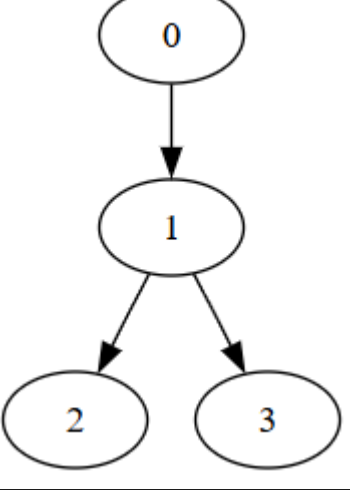
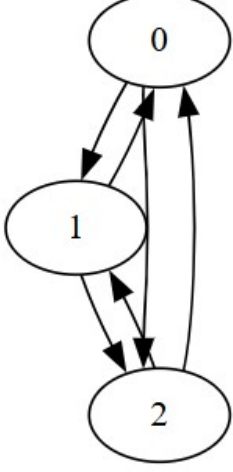
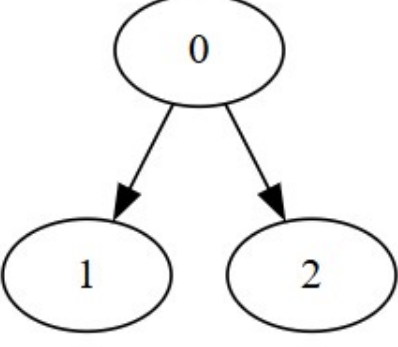
Приложение 3

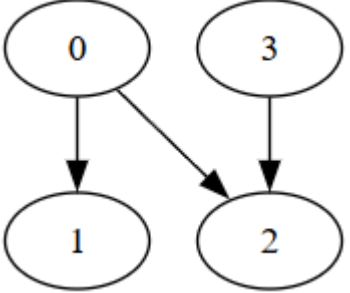
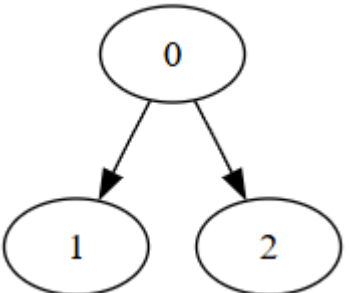
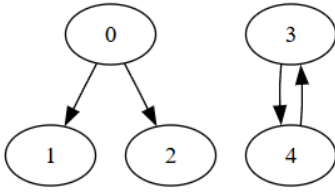
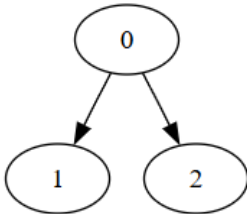
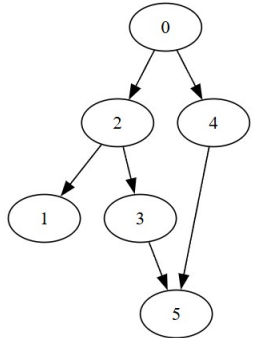
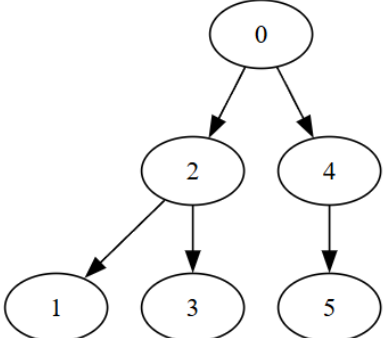
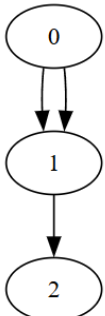
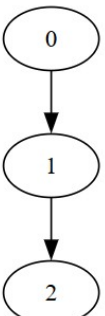
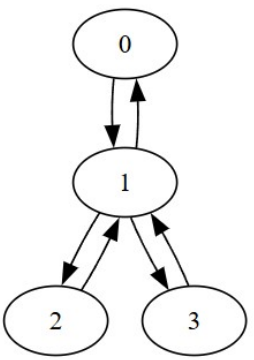
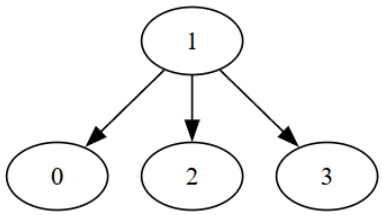
Методика тестирования функции DirGraph::createSpanningTree

```
/**
 * @brief создаёт строит остовное дерево графа, обходя его в ширину
 * @param rootVertex индекс корня дерева
 * @return ссылку на созданный граф
 */
DirGraph * DirGraph::createSpanningTree(int rootVertex);
```

№	Название	Входные данные	Входные данные	Ожидаемый результат
1	linearTree	0		

2	shrunkedLinearTree	1	 <pre> graph TD 0((0)) --> 1((1)) 1 --> 2((2)) 2 --> 3((3)) </pre>	 <pre> graph TD 1((1)) --> 2((2)) 2 --> 3((3)) </pre>
3	basicNonLinearTree	0	 <pre> graph TD 0((0)) --> 1((1)) 0 --> 2((2)) 0 --> 3((3)) </pre>	 <pre> graph TD 0((0)) --> 1((1)) 0 --> 2((2)) 0 --> 3((3)) </pre>
4	rootInDrain	2	 <pre> graph TD 0((0)) --> 1((1)) 0 --> 2((2)) 0 --> 3((3)) </pre>	 <pre> graph TD 2((2)) </pre>
5	severalRoots	1	 <pre> graph TD 0((0)) --> 2((2)) 1((1)) --> 2 2 --> 3((3)) 2 --> 4((4)) </pre>	 <pre> graph TD 1((1)) --> 2((2)) 2 --> 3((3)) 2 --> 4((4)) </pre>

6	simpleCycle	0		
7	containsCycle	0		
8	twoDirectionCycle	0		

9	unreachableVertices	0		
10	isolatedComponent	0		
11	searchOrderTest	0		
12	multigraph	0		
13	stronglyConnectedGraph	1		

14	simpleLoop	0	<pre> graph TD 0((0)) --> 1((1)) 1 --> 1 1 --> 2((2)) </pre>	<pre> graph TD 0((0)) --> 1((1)) 1 --> 2((2)) </pre>
15	ComplexLo ops	1	<pre> graph TD 0((0)) --> 0 0 --> 1((1)) 1 --> 1 1 --> 2((2)) 1 --> 3((3)) 2 --> 3 3 --> 1 </pre>	<pre> graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 3 --> 0((0)) </pre>
16	Complex Test1	0	<pre> graph TD 0((0)) --> 2((2)) 0 --> 1((1)) 0 --> 3((3)) 2 --> 6((6)) 6 --> 4((4)) 3 --> 1 3 --> 5((5)) 1 --> 5 10((10)) --> 11((11)) 11 --> 10 </pre>	<pre> graph TD 0((0)) --> 1((1)) 0 --> 2((2)) 0 --> 3((3)) 1 --> 4((4)) 1 --> 5((5)) 2 --> 6((6)) </pre>
17	Complex Test2	0	<pre> graph TD 7((7)) --> 8((8)) 8 --> 7 0((0)) --> 2((2)) 0 --> 1((1)) 0 --> 3((3)) 2 --> 6((6)) 1 --> 4((4)) 1 --> 5((5)) 3 --> 5 3 --> 3 </pre>	<pre> graph TD 0((0)) --> 1((1)) 0 --> 2((2)) 0 --> 3((3)) 1 --> 4((4)) 1 --> 5((5)) 2 --> 6((6)) </pre>

18	Complex Test3	5	<pre> graph TD 0((0)) --> 0 0 --> 2((2)) 2 --> 5((5)) 5 --> 5 5 --> 3((3)) 5 --> 4((4)) 3 --> 4 3 --> 6((6)) 4 --> 6 1((1)) </pre>	<pre> graph TD 5((5)) --> 3((3)) 5 --> 4((4)) 3 --> 6((6)) </pre>
19	Complex Test4	0	<pre> graph TD 0((0)) --> 1((1)) 0 --> 2((2)) 0 --> 3((3)) 2 --> 2 2 --> 5((5)) 5 --> 5 5 --> 3 5 --> 4((4)) 3 --> 4 3 --> 6((6)) 4 --> 6 </pre>	<pre> graph TD 0((0)) --> 1((1)) 0 --> 2((2)) 0 --> 3((3)) 2 --> 5((5)) 3 --> 4((4)) 3 --> 6((6)) </pre>

Приложение 4

Методика тестирования функции findGraphType

/**

* @brief определяет тип графа и находит дуги, при удалении которых из него можно сделать дерево (если можно)

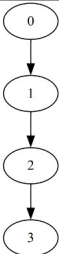
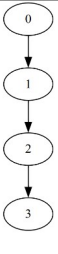
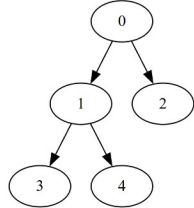
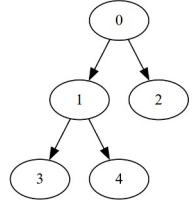
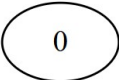
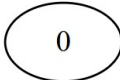
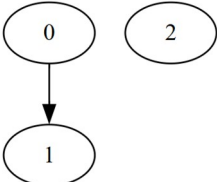
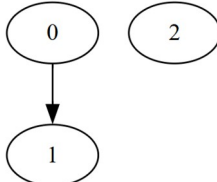
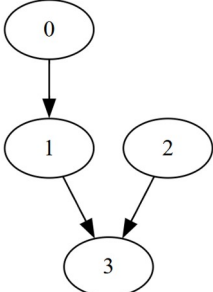
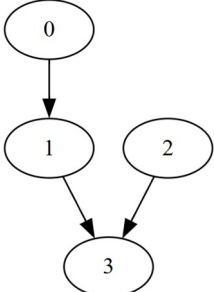
* @param graph константная ссылка на граф

* @param edgeDifference ссылка на набор дуг, который нужно удалить из графа

* @return тип дерева из перечисления graphType

*/

graphType findGraphType(const DirGraph & graph, adjacencyList & edgeDifference);

№	Название теста	Входные данные	Ожидаемый тип	Ожидаемый результат
1	linearTree		tree	
2	classicTree		tree	
3	oneVertexTree		tree	
4	isolatedVertex		notConvertibleToTree	
5	severalPotentialRoots		notConvertibleToTree	

6	onePotentialRoot		notConvertibleToTree	
7	noRoot		notConvertibleToTree	
8	simpleCycle		convertibleToTree	
9	excessEdge		convertibleToTree	

10	onePotential Root	<pre> graph TD 0((0)) --> 4((4)) 0((0)) --> 1((1)) 4((4)) --> 2((2)) 2((2)) --> 1((1)) 2((2)) --> 3((3)) 1((1)) --> 3((3)) </pre>	convertibleTo Tree	<pre> graph TD 0((0)) --> 4((4)) 0((0)) --> 1((1)) 4((4)) -.-> 2((2)) 2((2)) --> 1((1)) 2((2)) --> 3((3)) 1((1)) --> 3((3)) </pre>
11	noPotential Root	<pre> graph TD 0((0)) --> 4((4)) 0((0)) --> 1((1)) 4((4)) --> 2((2)) 2((2)) --> 1((1)) 2((2)) --> 3((3)) 1((1)) --> 3((3)) </pre>	Convertible ToTree	<pre> graph TD 0((0)) --> 4((4)) 0((0)) --> 1((1)) 4((4)) -.-> 2((2)) 2((2)) --> 1((1)) 2((2)) --> 3((3)) 1((1)) --> 3((3)) </pre>
12	multigraph	<pre> graph TD 0((0)) --> 4((4)) 0((0)) --> 1((1)) 4((4)) --> 2((2)) 2((2)) --> 1((1)) 2((2)) --> 3((3)) 1((1)) --> 3((3)) 1((1)) <--> 2((2)) </pre>	convertibleTo Tree	<pre> graph TD 0((0)) --> 4((4)) 0((0)) --> 1((1)) 4((4)) -.-> 2((2)) 2((2)) --> 1((1)) 2((2)) --> 3((3)) 1((1)) --> 3((3)) 1((1)) <--> 2((2)) </pre>
13	weakConnec tedCycles	<pre> graph TD 5((5)) --> 3((3)) 5((5)) --> 0((0)) 3((3)) --> 4((4)) 0((0)) --> 1((1)) 1((1)) --> 2((2)) 2((2)) --> 0((0)) </pre>	convertibleTo Tree	<pre> graph TD 5((5)) --> 3((3)) 5((5)) --> 0((0)) 3((3)) -.-> 4((4)) 0((0)) --> 1((1)) 1((1)) --> 2((2)) 2((2)) --> 0((0)) </pre>

14	strongConnectedCycles		convertibleTo Tree	
15	loops		convertibleTo Tree	
16	complexTest 1		convertibleTo Tree	
17	complexTest 2		convertibleTo Tree	